

LAB EXPERIMENT 07

OBJECTIVE: Design a data path of a computer from its register transfer language description.

EQUIPMENTS & COMPONENTS REQUIRED:

SL. No.	Equipment's	Specification	Quantity
1	Digital IC Trainer kit	-	1
2	Digital Multimeter		1

SL. No.	Components	Specification	Quantity
1	Digital ICs	7400, 7402, 7404, 7408, 7432, 7486.	1 each
2	Patch cords	-	6

BRIEF DESCRIPTION:

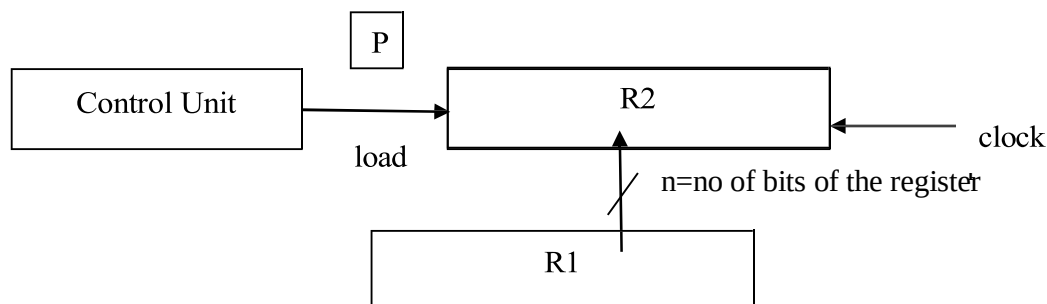
The symbolic notation used to describe the micro-operation transfers among registers is called Register Transfer Language. The term “register transfer” implies the availability of hardware logic circuits that can perform a stated micro-operation and transfer the result of the operation to the same or another register.

A statement that specifies a register transfer implies that circuits are available from the outputs of the source register to the inputs of the destination register and that the destination register has a parallel load capacity. If the transfer is to occur under a predetermined condition i.e.

If ($P=1$) then $R2 \leftarrow R1$

Where P is the control signal generated in the control section. A Control Function is a Boolean variable that is equal to 0 or 1

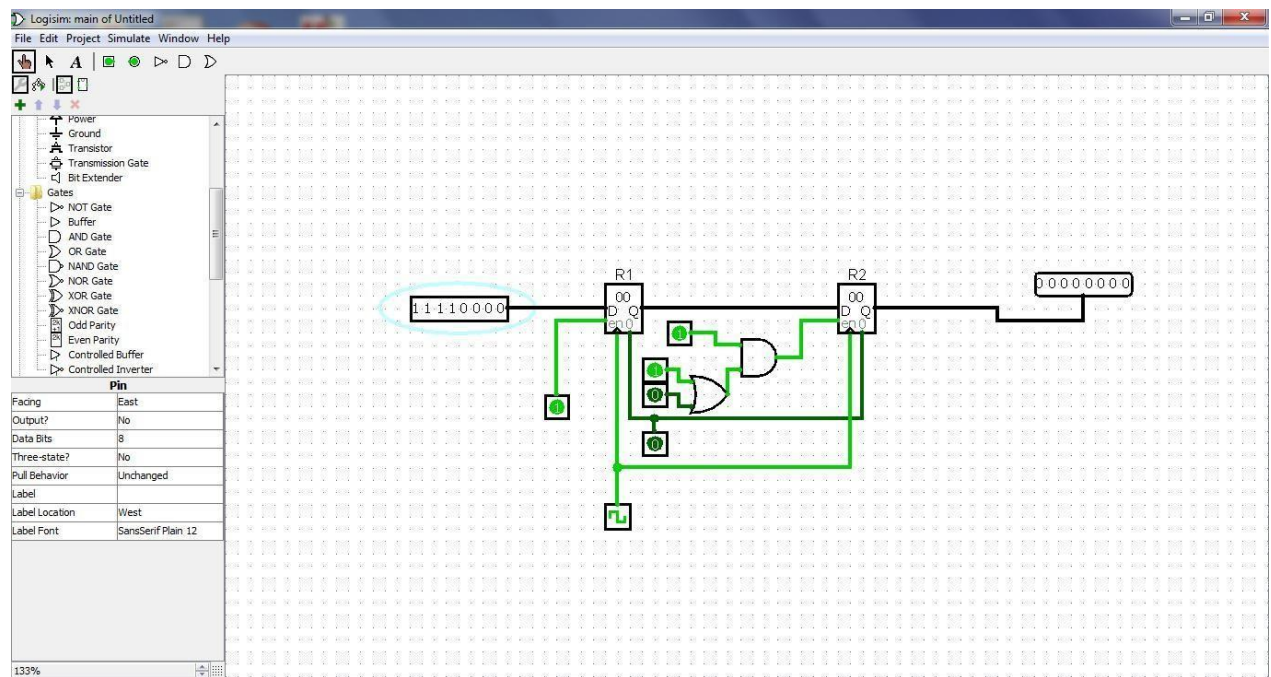
$P: R2 \leftarrow R1$



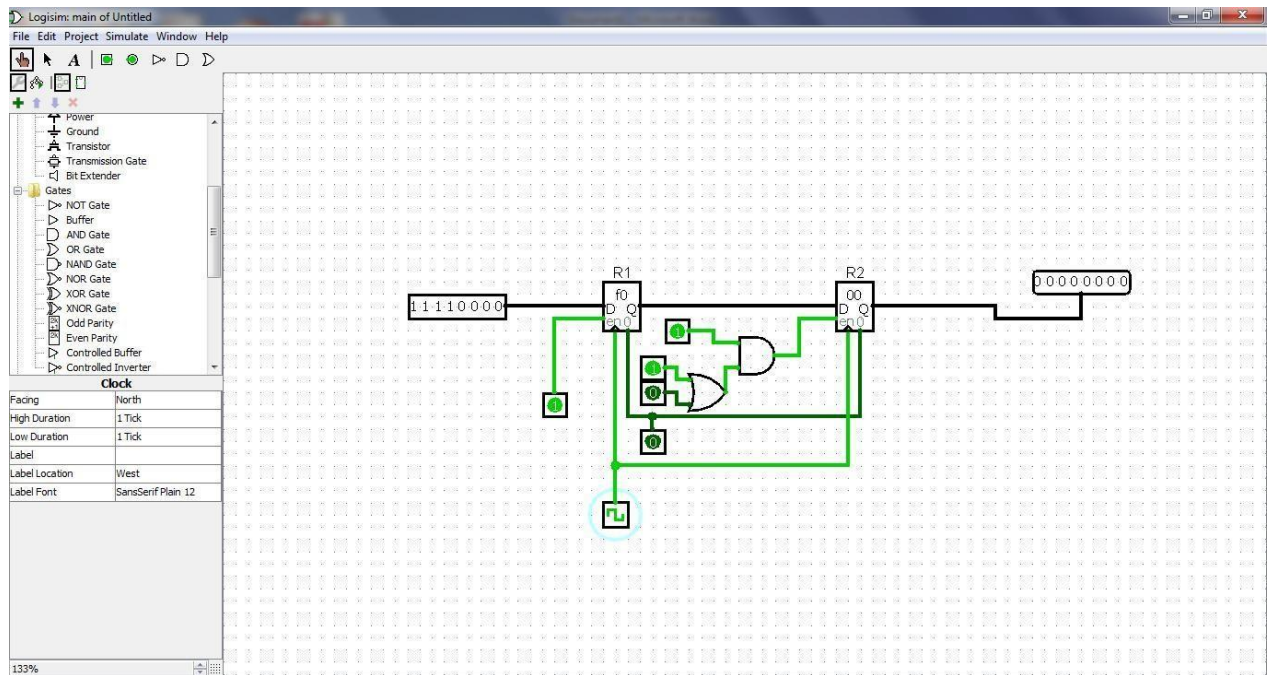
BLOCK DIAGRAM

PROCEDURE: -

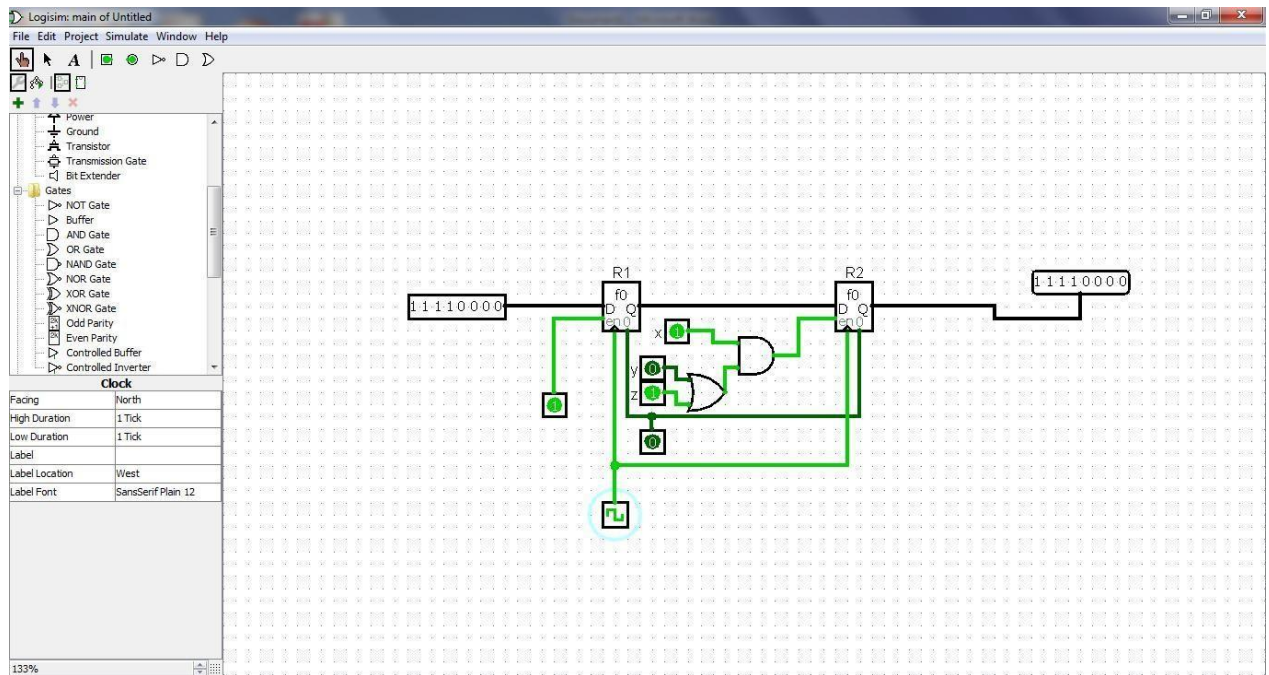
Data bits Ready to be transferred



In the first clock tick Register R1 stores the value



In the next clock pulse the value gets transferred to registerR2 if its Enable input is high, i.e. its control function $x.(y+z)=1$



Register Transfer using BUS

LAB EXPERIMENT 08

OBJECTIVE: Design the control unit of a computer using hardwired based on its register transfer language description.

COMPONENTS REQUIRED:

1. Registers
1. AND Gates
2. OR Gate
3. Memory
4. Decoders and Multiplexers
5. Connecting Wires etc.

BREIF DESCRIPTION:

The control unit (CU) is a component of a computer's central processing unit (CPU) that directs the operation of the processor. It tells the computer's memory, arithmetic/logic unit and input and output devices on how to respond to a program's instructions.

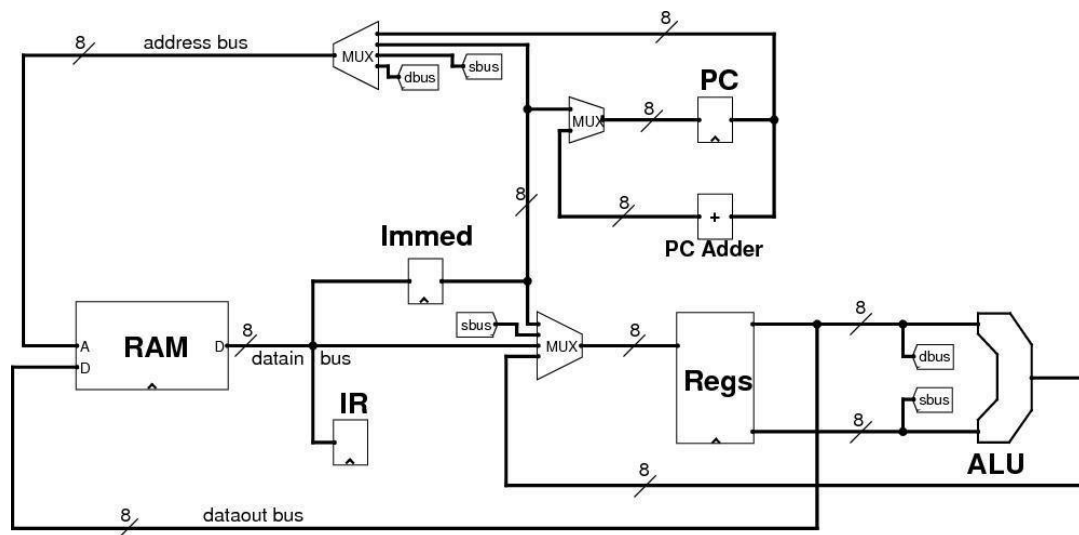
It directs the operation of the other units by providing timing and control signals. Most computer resources are managed by the CU. It directs the flow of data between the CPU and the other devices. The Control Unit (CU) is digital circuitry contained within the processor that coordinates the sequence of data movements into, out of, and between a processor's many sub-units. The result of these routed data movements through various digital circuits (sub-units) within the processor produces the manipulated data expected by a software instruction (loaded earlier, likely from memory). It controls (conducts) data flow inside the processor and additionally provides several external control signals to the rest of the computer to further direct data and instructions to/from processor external destinations (i.e. memory).

PROCEDURE:-

The CPU has an 8-bit data bus and an 8-bit address bus, so it can only support 256 bytes of memory to hold both instructions and data.

- Internally, there are four 8-bit registers, R0 to R3, plus an Instruction Register, the Program Counter, and an 8-bit register which holds immediate values.
 - The ALU : It performs the four operations AND, OR, ADD and SUB on two 8-bit values, and supports signed ADDs and SUBs.
 - The CPU is a load/store architecture: data has to be brought into registers for manipulation, as the ALU only reads from and writes back to the registers.
 - The ALU operations have two operands: one register is a source register, and the second register is both source and destination register, i.e. destination register = destination register OP source register.
-

- All the jump operations perform absolute jumps; there are no PC-relative branches. There are conditional jumps based on the zero or negativity of the destination register, as well as a "jump always" instruction.
- The following diagram shows the data paths in the CPU:

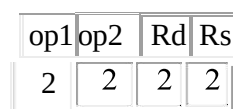


- The *dbus* and *sbus* labels indicate the lines coming out from the register file which hold the value of the destination and source registers.
- Note the data loop involving the registers and the ALU, whose output can only go back into a register.
- The data out bus is only connected to the *dbus* line, so the only value which can be written to memory is the destination register.
- Also note that there are only 3 multiplexors:
 - the address bus multiplexor can get a memory address from the PC, the immediate register (for direct addressing), or from the source or destination registers (for register indirect addressing).
 - The PC multiplexor either lets the PC increment, or jump to the value in the immediate register.
 - the multiplexor in front of the registers determines where a register write comes from: the ALU, the immediate register, another register or the data bus.

As we have decided the hardware to be used in our design now we have to decide the instruction set for our computer.

Instruction Set

- Half of the instructions in the instruction set fit into one byte:



- These instructions are identified by a 0 in the most-significant bit in the instruction, i.e. $op1=0X$.
- The 4bits of opcode are split into $op1$ and $op2$
- Rd is the destination register, and Rs is the source register.
- The other half of the instruction set are two-byte instructions. The first byte has the same format as above, and it is followed by an 8-bit constant or immediate value:

op1	op2	Rd	Rs	immediate
2	2	2	2	8

- These two-byte instructions are identified by a 1 in the most-significant bit in the instruction, i.e. $op1 = 1X$.
- With 4 operation bits, there are 16 instructions:

op1	op2	Mnemonic	Purpose
00	00	ANDRd, Rs	$Rd = Rd \text{ AND } Rs$
00	01	ORRd, Rs	$Rd = Rd \text{ OR } Rs$
00	10	ADDRd, Rs	$Rd = Rd + Rs$
00	11	SUBRd, Rs	$Rd = Rd - Rs$
01	00	LWRd, (Rs)	$Rd = \text{Mem}[Rs]$
01	01	SWRd, (Rs)	$\text{Mem}[Rs] = Rd$
01	10	MOVRd, Rs	$Rd = Rs$
01	11	NOP	Do nothing
10	00	JEQRd, immed	$PC = \text{immed}$ if $Rd == 0$
10	01	JNERd, immed	$PC = \text{immed}$ if $Rd \neq 0$
10	10	JGTRd, immed	$PC = \text{immed}$ if $Rd > 0$
10	11	JLTRd, immed	$PC = \text{immed}$ if $Rd < 0$
11	00	LWRd, immed	$Rd = \text{Mem}[\text{immed}]$
11	01	SWRd, immed	$\text{Mem}[\text{immed}] = Rd$
11	10	LIRd, immed	$Rd = \text{immed}$
11	11	JMPimmed	$PC = \text{immed}$

- Note the regularity of the ALU operations and the jump operations: we can feed the $op2$ bits directly into the ALU, and use $op2$ to control the branch decision.
- The rest of the instruction set is less regular, which will require special decoding for certain of the 16 instructions.

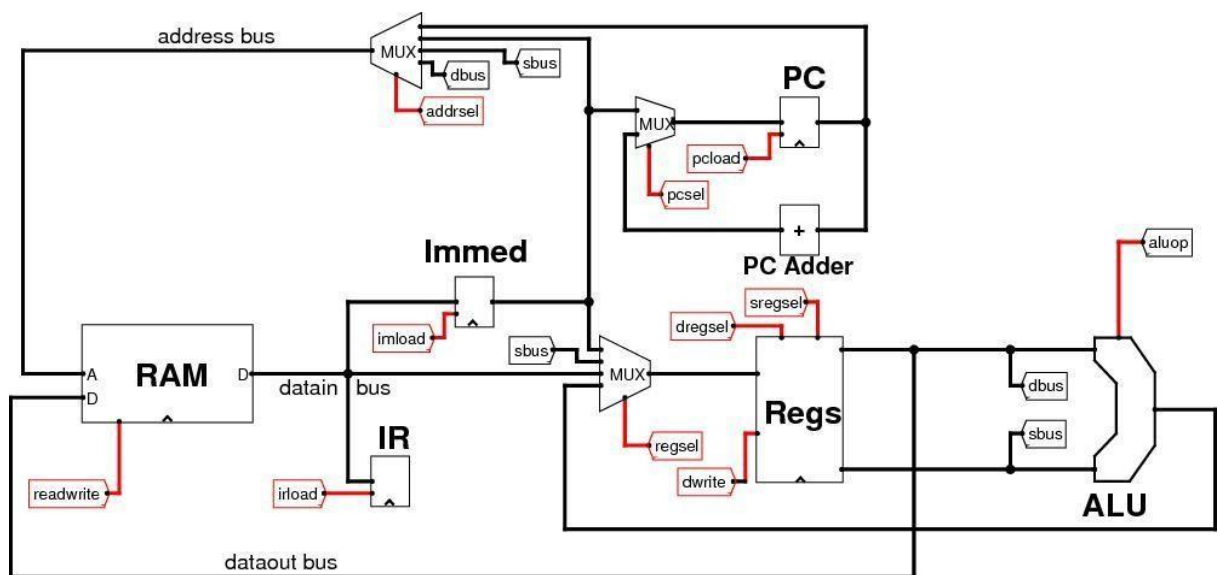
Instruction Phases

- The CPU internally has three phases for the execution of each instruction.

- On phase 0, the instruction is fetched from memory and stored in the Instruction Register.
- On phase 1, if the fetched instruction is a two-byte instruction, the second byte is fetched from memory and stored in the Immediate Register. For one-byte instructions, nothing occurs in phase 1.
- On phase 2, everything else is done as required, which can include:
 - An ALU operation, reading from two registers.
 - A jump decision which updates the PC.
 - A register write.
 - A read from a memory location.
 - A write to a memory location.
- After phase 2, the CPU starts the next instruction in phase 0.

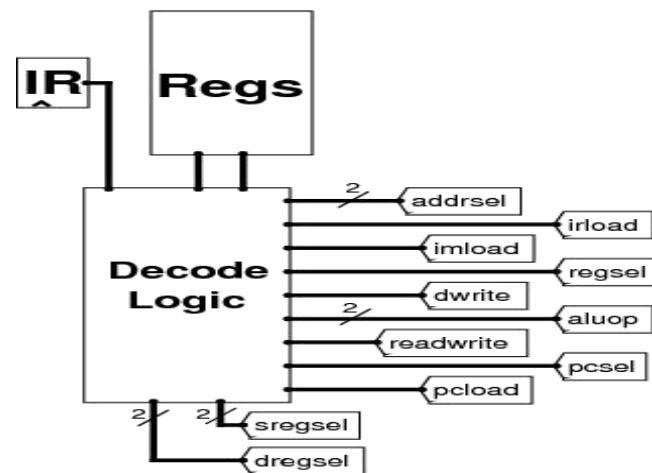
CPU Control Lines

- Below is the main CPU diagram again, this time with the control lines shown.



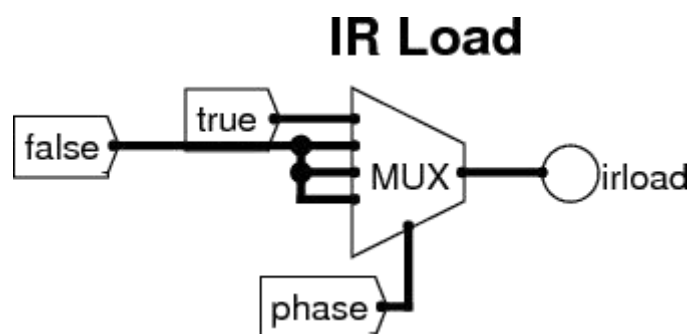
- There are several 1-bit control lines:
 - *pcsel*, increment PC or load the jump value from the Immediate Register.
 - *pload*, load the PC with a new value, or don't load a new value.
 - *irload*, load the Instruction Register with a new instruction.
 - *imload*, load the Immediate Register with a new value.
 - *readwrite*, read from memory, or write to memory.
 - *dwrite*, write a value back to a register, or don't write a value.
- There are also several 2-bit control lines:
 - *addrsel*, select an address from the PC, the Immediate Register, the source register or the destination register.
 - *regsel*, select a value to write to a register from the Immediate Register, another register, the data bus or from the ALU.
 - *Dregsel* and *sregsel*, select two registers whose values are sent to the ALU.
 - *aluop*, which are the op2 bits that control the operation of the ALU.

- The values for all of these control lines are generated by the Decode Logic, which gets as input the value from the Instruction Register, and the zero & negative lines of the destination register.



Phase Zero

- On phase zero, the PC's value has to be placed on the address bus, so the *addrsel* line must be 0. The *irload* line needs to be 1 so that the IR is loaded from the *data in* bus. Finally, the PC must be incremented in case we need to fetch an immediate value in phase 1.
- All of this can be done using multiplexors which output different values depending on the current phase. Here is the control logic for the *irload* line.



- We only need to load the IR on phase 0, so we can wire true to the 0 input of the *irload* multiplexer, and false to the other inputs. **Note:** input 1 (i.e. decimal 3) to the multiplexer is never used, as we never get to phase 3, but Logisim wants all multiplexer inputs to be valid.
- Another way to look at each phase is the value which needs to be set for each control line, for each instruction.
- For phase zero, these control line values can be set for all instructions:

op	op	instru	pcse	pcloa	Irloa	imloa	R	dwrite	jumps	addrs	regs	dre	sre	aluo
1	2	ct	l	d	d	d	w	e	el	el	el	g	g	p

0xxx	all	x	0	0	0	0	0	0	0	0	x	X	x	x
1xxx	all	1	1	0	1	0	0	0	0	0	x	X	x	x

Phase Two

- Here, the values of the control lines depend heavily on what specific instruction we are performing. Here's the table of control line outputs depending on the instruction:

op1	op2	instruct	pcsel	pcload	irload	imload	rw	dwrite	addrsel	regsel	dreg	sreg	aluop
00	00	ANDRd,Rs	x	0	0	0	0	1	x	3	Rd	Rs	op2
00	01	ORRd, Rs	x	0	0	0	0	1	x	3	Rd	Rs	op2
00	10	ADDRd,Rs	x	0	0	0	0	1	x	3	Rd	Rs	op2
00	11	SUBRd,Rs	x	0	0	0	0	1	x	3	Rd	Rs	op2
01	00	LWRd,(Rs)	x	0	0	0	0	1	2	2	Rd	Rs	x
01	01	SWRd,(Rs)	x	0	0	0	1	0	3	x	Rd	Rs	x
01	10	MOVRd, Rs	x	0	0	0	0	1	x	1	Rd	Rs	x
01	11	NOP	x	0	0	0	0	0	x	x	x	x	x
10	00	JEQRd,immed	0	J	0	0	0	0	x	x	Rd	x	op2
10	01	JNERd,immed	0	J	0	0	0	0	x	x	Rd	x	op2
10	10	JGTRd,immed	0	J	0	0	0	0	x	x	Rd	x	op2
10	11	JLTRd,immed	0	J	0	0	0	0	x	x	Rd	x	op2
11	00	LWRd,immed	x	0	0	0	0	1	1	2	Rd	x	x
11	01	SWRd,immed	x	0	0	0	1	0	1	x	Rd	x	x
11	10	LIRd,immed	x	0	0	0	0	1	x	0	Rd	x	x
11	11	JMPimmed	0	1	0	0	0	0	x	x	x	x	x

- To make the control line logic as simple as possible, a CPU designer is always striving for regularity. However, this is often in conflict with the desired CPU functionality.
- From the table above, the ALU instructions ($op1=00$) and the jump instructions ($op1=10$) are nice and regular. All the $op1=1x$ instructions use the Immediate Register, while the $op1=0x$ instructions don't.
- We can always tie $dregsel$ to Rd from the instruction, and the same goes for $sregsel=Rd$ and $aluop=op2$. And $irload$ and $imload$ are always 0 for phase 2.
- With the remaining control lines, the regularities cease.

Putting this all back together, we now have this device:

